CHAPTER 04: TEST ANALYSIS AND DESIGN (පරීක්ෂණ විශ්ලේෂණය සහ සැලසුම්කරණය)

1. Core Syllabus Concepts & In-Depth Notes (ප්‍රධාන විෂය කරුණු විස්තරාත්මකව)

4.1 Test Techniques Overview FL-4.1.1

Test techniques මගින් tester හට **Test Analysis ("WHAT to test")** සහ **Test Design ("HOW to test")** යන කාර්යයන් ක්‍රමානුකූලව හා විද්‍යාත්මකව සිදුකිරීමට මග පෙන්වයි. ප්‍රධාන කාණ්ඩ 3කි:

1. Black-Box Techniques:	2. White-Box Techniques:	3. Experience-Based:
Specification-based: අභ්‍යන්තර code එක නොබලා formal requirements, user stories, සහ models මත පදනම්ව test conditions සහ test cases සාදයි.	**Structure-based:** අභ්‍යන්තර කේතයේ structure, architecture, statements, සහ decision branches මත පදනම්ව test cases සාදයි.	පරීක්ෂකයාගේ අත්දැකීම් (experience), බුද්ධිය, සමාන පද්ධති වල අතීත දෝෂ සහ පරිශීලක හැසිරීම් මත පදනම්ව පරීක්ෂා කරයි.

1.2 Black-Box Test Techniques (කළු-පෙට්ටි පරීක්ෂණ ශිල්පක්‍රම) FL-4.2 (K3)

1. Equivalence Partitioning (EP - සමතුල්‍යතා කලාපකරණය) FL-4.2.1

පද්ධතිය එකම ආකාරයකින් සලකනු ලබන (processed in the same way) දත්ත කාණ්ඩ වලට බෙදයි. සෑම partition එකකින්ම එක් අගයක් පරීක්ෂා කිරීමෙන් මුළු partition එකම නියෝජනය වේ:

- **Valid Partitions:** පද්ධතිය විසින් පිළිගන්නා (valid inputs) නිවැරදි අගයන්.
- **Invalid Partitions:** පද්ධතිය විසින් ප්‍රතික්ෂේප කළ යුතු (invalid inputs) වැරදි අගයන්.

```
EP Coverage = (Exercised Partitions / Total Identified Partitions) × 100%
```

2. Boundary Value Analysis (BVA - සීමා අගය විශ්ලේෂණය) FL-4.2.2

බොහෝ දෝෂ හටගන්නේ partitions වල මායිම් (boundaries) වලදීය (e.g. `<` වෙනුවට `<=` යෙදීම). BVA වර්ග 2ක් විභාගයේදී පරීක්ෂා කෙරේ:

2-Value BVA (මායිමකට අගයන් 2ක්):
සෑම boundary එකකටම අගයන් 2ක් ගනී: **Boundary value එක** සහ ඊට යාබද **අසල්වැසි partition එකේ ආසන්නතම අගය**.
උදා: වයස [18 සිට 65] දක්වා නම්:
• Lower boundary (18): **17** (invalid), **18** (valid)
• Upper boundary (65): **65** (valid), **66** (invalid)
එකතුව = අගයන් 4ක්: {17, 18, 65, 66}.

3-Value BVA (මායිමකට අගයන් 3ක්):
සෑම boundary එකකටම අගයන් 3ක් ගනී: **Boundary value එක** සහ එහි **දෙපසම ඇති ආසන්නතම අගයන් 2** (Both immediate neighbors).
උදා: Boundary 18 සඳහා:
• Left neighbor: **17**
• Boundary itself: **18**
• Right neighbor: **19**
එකතුව = {17, 18, 19}.

3. Decision Table Testing (තීරණ වගු පරීක්ෂාව) FL-4.2.3

සංකීර්ණ ව්‍යාපාරික රීති (Business Rules) සහ කොන්දේසි සංයෝජන (condition combinations) පරීක්ෂා කිරීමට යොදාගනී. Binary කොන්දේසි (True/False) n සංඛ්‍යාවක් ඇති විට මුළු රීති (Rules) ගණන 2^n වේ.

• **Collapsing / Minimization:** යම් කොන්දේසියක අගය කුමක් වුවත් action එක වෙනස් නොවේ නම්, එය "Don't Care" ($-$) ලෙස සලකා columns ඒකාබද්ධ කළ හැක.

4. State Transition Testing (තත්ත්ව සංක්‍රාන්ති පරීක්ෂාව) FL-4.2.4

පද්ධතිය විවිධ අවස්ථා (states) වල පවතින විට සහ සිදුවීම් (events / inputs) නිසා තත්ත්වයෙන් තත්ත්වයට මාරුවන විට (transitions) භාවිත කරයි.

• **0-Switch Coverage (Valid Transitions Coverage):** පද්ධතියේ ඇති සියලුම තනි වලංගු transitions එක් වරක් හෝ execute කිරීම.

• **Invalid Transitions:** යම් state එකකදී සිදු නොවිය යුතු සිදුවීමක් වූ විට පද්ධතිය නිවැරදිව error message එකක් පෙන්වන්නේදැයි බැලීම.

1.3 White-Box Test Techniques (සුදු-පෙට්ටි පරීක්ෂණ ශිල්පක්‍රම) FL-4.3

Statement Testing & Coverage FL-4.3.1 **:**

කේතයේ ඇති executable statements කොපමණ ප්‍රමාණයක් execute වී ඇත්දැයි මනියි:

```
Statement Cov = (Executed Statements / Total Executable Statements) × 100%
```

Branch Testing & Coverage FL-4.3.2 **:**

Control flow එකේ ඇති decision outcomes (e.g. `if-then-else` හි True සහ False ශාඛා) execute වී ඇත්දැයි මනියි:

```
Branch Cov = (Executed Branches / Total Decision Outcomes) × 100%
```

The Branch Subsumption Law (ශාඛා ආවරණ නීතිය):

- **100% Branch Coverage ලබා ගැනීමෙන් 100% Statement Coverage ස්වයංක්‍රීයවම තහවුරු වේ** (Branch coverage subsumes statement coverage).
- නමුත් **100% Statement Coverage ලැබූ පමණින් 100% Branch Coverage තහවුරු නොවේ!** (උදා: `else` කොටසක් නැති `if` statement එකක True ශාඛාව පමණක් ගිය විට Statement coverage 100% වුවද Branch coverage 50% කි).
- **White-box වල ප්‍රධාන සීමාව (Limitation):** කේතයේ කිසිදා නොලියූ, අතපසු වූ requirements (**Defects of Omission**) White-box පරීක්ෂාවෙන් කිසිසේත්ම සොයාගත නොහැක!

1.4 Experience-Based Test Techniques (අත්දැකීම් පාදක පරීක්ෂණ ශිල්පක්‍රම) FL-4.4

පරීක්ෂකයාගේ (Tester), සංවර්ධකයාගේ (Developer) සහ පරිශීලකයාගේ අත්දැකීම්, පද්ධතිය පිළිබඳ පූර්ව දැනුම, සහ වැරදි සිදුවිය හැකි ස්ථාන පිළිබඳ බුද්ධිමය තක්සේරුව මත පදනම්ව සිදුකරන පරීක්ෂණ ක්‍රමවේද වේ.

4.4.1 Error Guessing (දෝෂ අනුමාන කිරීම) FL-4.4.1 (K2)

දෝෂ අනුමාන කිරීමේ පදනම (Basis of Error Guessing):

සංවර්ධකයින් අතින් අතීතයේදී සිදුවූ වැරදි (mistakes), පද්ධතිවල හමුවූ දෝෂ (defects), සහ බිඳවැටීම් (failures) පිළිබඳ දැනුම මත පදනම්ව අනාගතයේ සිදුවිය හැකි දෝෂ කල්තියා අනුමාන කිරීමයි.

- *Fault Attack Lists:* බහුලව සිදුවන වැරදි ලැයිස්තුගත කර (e.g. Divide by zero, Blank inputs, Special characters, Null pointers, Session timeouts) පද්ධතියට පහරදීම.

4.4.2 Exploratory Testing (ගවේෂණාත්මක පරීක්ෂාව) FL-4.4.2 (K2)

Test Design, Execution, සහ Learning එකවර සිදුවීම:

කලින් ලියන ලද විධිමත් test cases නොමැතිව, පද්ධතිය ක්‍රියාත්මක කරමින් ඉගෙන ගන්නා අතරතුරදීම test cases නිර්මාණය කර ධාවනය කිරීම.

- **Test Charter:** පරීක්ෂාව මෙහෙයවීම සඳහා ඉලක්ක, විෂය පථය සහ උපදෙස් ඇතුළත් ලියවිල්ලකි.
- **Timeboxing:** සාමාන්‍යයෙන් පැය 1 ත් 2 ත් අතර නොබිඳුණු කාල පරිච්ඡේදයක් (Session-based testing) තුළ අවධානය යොමුකර පරීක්ෂා කිරීම.

4.4.3 Checklist-Based Testing (පිරික්සුම් ලැයිස්තු පදනම් කරගත් පරීක්ෂාව) FL-4.4.3 (K2)

පිරික්සුම් ලැයිස්තුවක් (Checklist) භාවිතය:

පරීක්ෂා කළ යුතු කරුණු, අත්දැකීම්, පරිශීලකයාට වැදගත් වන දෑ, හෝ නීතිමය ප්‍රමිතීන් ඇතුළත් පිරික්සුම් ලැයිස්තුවක් (Checklist) පදනම් කරගෙන පරීක්ෂණ සැලසුම් කිරීම හා ධාවනය කිරීම.

- ඉහළ මට්ටමේ මඟපෙන්වීමක් සපයන අතර, සවිස්තර පියවර නැති බැවින් ටෙස්ටර්ට නිදහසේ පරීක්ෂා කිරීමට ඉඩ ලැබේ.

1.5 Collaboration-Based Test Approaches (සහයෝගීතා පරීක්ෂණ ප්‍රවේශයන්) FL-4.5

4.5.1 Collaborative User Story Writing (සාමූහික User Stories ලිවීම) FL-4.5.1 (K2)

User Stories ලියනු ලබන්නේ තනි පුද්ගලයෙකු විසින් නොව, **Business Representatives, Developers, සහ Testers යන පාර්ශ්ව 3 හි සාමූහික එකමුතුවෙනි (The Three Amigos)**.

The 3 C's Concept (Ron Jeffries):

- **1. Card:** User Story එකේ සාරාංශය ලියන භෞතික හෝ ඉලෙක්ට්‍රොනික කාඩ්පත (e.g. "As a [role], I want [feature], so that [benefit]").
- **2. Conversation:** අවශ්‍යතාවය පිළිබඳ විස්තර සාකච්ඡා කර අවබෝධය හුවමාරු කරගැනීම.
- **3. Confirmation:** කතාව අවසන් බව තහවුරු කරන **Acceptance Criteria** වේ.

4.5.2 Acceptance Criteria (පිළිගැනීමේ නිර්ණායක) FL-4.5.2 (K2)

මෘදුකාංගය නිවැරදිව නිමකළ බව සහ පරිශීලකයා විසින් භාරගැනීමට සුදුසු බව තහවුරු කරන කොන්දේසි මාලාවකි. ප්‍රධාන ක්‍රම 2කට ලියනු ලැබේ:

1. Scenario-Oriented (Gherkin):

`Given` [පූර්ව කොන්දේසිය],
`When` [සිදුවීම හෝ ක්‍රියාව],
`Then` [අපේක්ෂිත ප්‍රතිඵලය].
BDD/ATDD ස්වයංක්‍රීයකරණයට ඉතා පහසුය.

2. Rule-Oriented (රීති පාදක):

වගු (tables) හෝ බුලට් පොයින්ට්ස් (bulleted lists) ලෙස ව්‍යාපාරික රීති ලැයිස්තුගත කිරීම.
උදා: "මුරපදයේ අකුරු 8ක් තිබිය යුතුය", "අවම එක් ඉලක්කමක් තිබිය යුතුය".

4.5.3 Acceptance Test-Driven Development (ATDD) `FL-4.5.3 (K3)`

ATDD වලදී Test Cases සෑදිය යුතු නිල අනුපිළිවෙල (Derivation Order):

1. **1. Positive Tests (මුලින්ම):** ප්‍රධාන සාර්ථක ප්‍රවාහය (Happy path) සහ මූලික පිළිගැනීමේ නිර්ණායක තහවුරු කිරීම.
2. **2. Negative Tests (දෙවනුව):** වැරදි inputs, boundary violations, සහ exception handling ආවරණය කිරීම.
3. **3. Non-Functional Tests (අවසානයේ):** Performance, usability, සහ security ලක්ෂණ ආවරණය වන පරිදි tests සකස් කිරීම.

3. Complete Chapter 4 Keyword Lexicon (නිල පාරිභාෂික ශබ්දකෝෂය)

Acceptance Criteria

මෘදුකාංගය පරිශීලකයා විසින් පිළිගැනීමට සපුරාලිය යුතු නිල කොන්දේසි.

ATDD

Acceptance criteria මත පදනම්ව සංවර්ධනයට පෙර tests නිර්මාණය කරන සහයෝගීතා ක්‍රමය.

Black-Box Test Technique

කේතයේ අභ්‍යන්තර ව්‍යුහය නොදැන specifications මත පමණක් පදනම්ව කරන පරීක්ෂණ ක්‍රමය.

Boundary Value Analysis (BVA)

සීමාන්තික අගයන් (boundaries) ඉලක්ක කරගෙන 2-value හෝ 3-value ලෙස කරන පරීක්ෂාව.

Branch Coverage

කේතයේ decision outcomes (True/False ශාඛා) ආවරණය වීමේ ප්‍රතිශතය.

Checklist-Based Testing

පිරික්සුම් ලැයිස්තුවක ඇති කරුණු ආවරණය වන පරිදි සිදුකරන පරීක්ෂාව.

Collaboration-Based Test Approach

Developers, Testers සහ Business Reps එකමුතුවෙන් test cases සාදන ප්‍රවේශය.

Coverage

පරීක්ෂණ මඟින් ආවරණය කරන ලද අයිතමයන්ගේ ප්‍රමාණාත්මක ප්‍රතිශතය.

Coverage Item

ආවරණය ගණනය කිරීම සඳහා පදනම් කරගන්නා නිශ්චිත ව්‍යුහය හෝ අවශ්‍යතාව (statement, branch, etc.).

Decision Table Testing

කොන්දේසි සංයෝජන සහ ව්‍යාපාරික රීති වගුවක් ලෙස නිරූපණය කර කරන පරීක්ෂාව.

Equivalence Partitioning (EP)

ආදාන දත්ත වලංගු සහ වලංගු නොවන කොටස් වලට බෙදා එක් අගයක් තෝරාගැනීම.

Error Guessing

ටෙස්ටර්ගේ අත්දැකීම් මත පදනම්ව සංවර්ධක වැරදි අනුමාන කර පරීක්ෂා කිරීම.

Experience-Based Test Technique

ටෙස්ටර්ගේ අත්දැකීම් සහ බුද්ධිමය තක්සේරුව පදනම් කරගත් පරීක්ෂණ ක්‍රම.

Exploratory Testing

Test charter සහ timebox භාවිතයෙන් design, execution සහ learning එකවර සිදුකිරීම.

State Transition Testing

තත්ත්ව සටහනක් (state diagram) ඇසුරින් transitions සහ states පරීක්ෂා කිරීම.

Statement Coverage

කේතයේ ක්‍රියාත්මක කළ හැකි ප්‍රකාශ (statements) ආවරණය වීමේ ප්‍රතිශතය.

Test Technique

Test conditions, test cases, සහ test data සෑදීමට භාවිත කරන විධිමත් ක්‍රමය.

White-Box Test Technique

මූල කේතයේ අභ්‍යන්තර ව්‍යුහය (control flow) මත පදනම්ව කරන පරීක්ෂණ ක්‍රම.

STUDENT NOTES & EXAM FLASHCARD REMINDERS (සිසු සටහන් සඳහා ඉඩ)

4. Realistic ISTQB Practice Exam Questions & Detailed Rationales (අභ්‍යාස ප්‍රශ්න සහ විවරණ)

Question 1 [K3 - FL-4.2.1]: An insurance application accepts an applicant's age as an integer input. Valid ages for coverage are from 18 to 65 inclusive. Ages under 18 are rejected with "Too young", and ages over 65 are rejected with "Too old". Inputs must be positive integers up to 120. Using Equivalence Partitioning, what is the MINIMUM number of test cases required to achieve 100% equivalence partition coverage?

(රක්ෂණ යෙදුමක වයස 18 සිට 65 දක්වා වලංගු වේ. 18ට අඩු අගයන් "Too young" ලෙසද, 65ට වැඩි අගයන් "Too old" ලෙසද ප්‍රතික්ෂේප වේ (උපරිම 120 දක්වා). Equivalence Partitioning ක්‍රමයට අනුව 100% coverage ලබාගැනීමට අවශ්‍ය අවම Test cases ගණන කීයද?)

A) 2 test cases

B) 3 test cases

C) 4 test cases

D) 6 test cases

Correct Answer: B

Rationale (විවරණය): ආදාන පරාසය කොටස් 3කට බෙදේ:

1. Invalid Partition: 1 සිට 17 දක්වා ("Too young")
2. Valid Partition: 18 සිට 65 දක්වා ("Accepted")
3. Invalid Partition: 66 සිට 120 දක්වා ("Too old")

100% EP coverage ලබා ගැනීමට එක් එක් partition එකෙන් එක් අගය බැගින් **අවම වශයෙන් test cases 3ක්** අවශ්‍ය වේ (e.g. 10, 30, 70).

Question 2 [K3 - FL-4.2.2]: For the age requirement above (valid range 18 to 65 inclusive), which set of test values achieves 100% coverage using 2-VALUE Boundary Value Analysis (BVA)?

(ඉහත වයස් සීමාව සඳහා (18 සිට 65 දක්වා වලංගු වේ), 2-Value Boundary Value Analysis (BVA) ක්‍රමවේදය භාවිතයෙන් 100% ආවරණයක් ලබාදෙන පරීක්ෂණ අගයන් කට්ටලය කුමක්ද?)

A) 17, 18, 65, 66

B) 17, 18, 19, 64, 65, 66

C) 18, 65

D) 0, 18, 65, 120

Correct Answer: A

Rationale (විවරණය): 2-value BVA වලදී එක් මායිමකට අගයන් 2 බැගින් ගනී:

• Lower boundary (18): 18 (valid) සහ 17 (invalid).

• Upper boundary (65): 65 (valid) සහ 66 (invalid).

එබැවින් **{17, 18, 65, 66}** මගින් 100% 2-value BVA coverage ලැබේ.

Question 3 [K3 - FL-4.2.2]: Which set of test values represents 3-VALUE Boundary Value Analysis for the lower boundary (18) of the valid range [18 to 65]?

(18) of the valid range [18 to 65]? ([18 සිට 65] වලංගු පරාසයේ පහළ මායිම (18) සඳහා 3-Value Boundary Value Analysis (BVA) අගයන් නියෝජනය කරන්නේ කුමන අගයන් කට්ටලයද?)

A) 17 and 18

B) 17, 18, and 19

C) 16, 17, and 18

D) 18, 19, and 20

Correct Answer: B

Rationale (විවරණය): 3-value BVA වලදී මායිම් අගය සහ එහි දෙපසම අගයන් ගනී. 18 සඳහා: වම්පස අසල්වැසියා = 17, මායිම = 18, දකුණුපස අසල්වැසියා = 19 වේ. එබැවින් **{17, 18, 19}** වේ.

Question 4 [K2 - FL-4.3.2]: A program contains a single `if...then` decision statement with no `else` block. A single test case executes the condition such that it evaluates to TRUE, running the inner code. What are the resulting Statement Coverage and Branch Coverage percentages?

(ක්‍රමලේඛයක else කොටසක් නොමැති තනි if...then තීරණයක් ඇත. තනි test case එකකින් කොන්දේසිය TRUE වී අභ්‍යන්තර කේතය ක්‍රියාත්මක වේ නම්, Statement Coverage සහ Branch Coverage ප්‍රතිශත පිළිවෙළින් කොපමණද?)

A) 100% Statement Coverage, 100% Branch Coverage

B) 100% Statement Coverage, 50% Branch Coverage

C) 50% Statement Coverage, 50% Branch Coverage

D) 50% Statement Coverage, 100% Branch Coverage

Correct Answer: B

Rationale (විවරණය): Inner code එකේ ඇති සියලු statements execute වූ බැවින් Statement Coverage **100%** වේ. නමුත් `if` condition එකකට True සහ False ලෙස ශාඛා (branches) 2ක් ඇත. මෙහි execute වූයේ True ශාඛාව පමණක් බැවින් Branch Coverage 1/2 හෙවත් **50%** කි.

Question 5 [K3 - FL-4.2.4]: A digital banking account has three states: [Open], [Overdrawn], and [Suspended]. The system allows valid transitions: Open → Overdrawn, Overdrawn → Open, Overdrawn → Suspended, and Suspended → Open. What is the MINIMUM number of valid transitions that must be exercised to achieve 100% Valid Transitions Coverage (0-Switch)?

(ඩිජිටල් බැංකු ගිණුමක [Open], [Overdrawn], සහ [Suspended] යන තත්ත්ව 3ක් ඇත. වලංගු සංක්‍රාන්ති (Transitions) 4ක් පවතී: Open -> Overdrawn, Overdrawn -> Open, Overdrawn -> Suspended, සහ Suspended -> Open. 100% Valid Transitions Coverage (0-Switch) ලබාගැනීමට පරීක්ෂා කළ යුතු අවම සංක්‍රාන්ති සංඛ්‍යාව කීයද?)

A) 3 transitions

B) 4 transitions

C) 6 transitions

D) 9 transitions

Correct Answer: B

Rationale (විවරණය): 0-switch coverage (Valid Transitions coverage) මගින් සියලුම වලංගු single transitions ආවරණය කළ යුතුය. මෙහි වලංගු transitions හරියටම 4ක් ඇති බැවින් (Open→Overdrawn, Overdrawn→Open, Overdrawn→Suspended, Suspended→Open), ඒ **4ම** execute කළ යුතුය.

Question 6 [K2 - FL-4.3.3]: What is a primary limitation of white-box test techniques?

(ශ්වේත-පෙට්ටි පරීක්ෂණ තාක්ෂණයේ (White-box test techniques) ප්‍රධාන සීමාවක් හෝ අවාසියක් වන්නේ කුමක්ද?)

A) They cannot provide an objective measurement of code coverage.

B) They cannot detect defects of omission where requirements were never implemented in the code.

C) They cannot be applied during static code reviews or dry runs.

D) They are independent of how the software is designed or implemented.

Correct Answer: B

Rationale (විවරණය): White-box techniques පදනම් වන්නේ ලියා ඇති source code එක මතය. එබැවින් developer විසින් කිසිසේත්ම කේතය නොලියා අතපසු කළ requirements (**defects of omission**) White-box පරීක්ෂාවෙන් හඳුනාගත නොහැක.

Question 7 [K2 - FL-4.4.2]: In session-based exploratory testing, what document provides the tester with high-level test objectives to guide their time-boxed exploration?

(Session-based exploratory testing හිදී, කාල සීමා සහිත ගවේෂණය මෙහෙයවීම සඳහා ටෙස්ටර්ට ඉහළ මට්ටමේ පරීක්ෂණ අරමුණු සපයන ලේඛනය කුමක්ද?)

A) Detailed Test Procedure

B) Test Charter

C) Decision Table

D) Traceability Matrix

Correct Answer: B

Rationale (විවරණය): Exploratory testing වලදී tester හට පරීක්ෂා කළ යුතු ඉලක්ක සහ සීමාවන් ලබාදෙන ලියවිල්ල **Test Charter** එකක් ලෙස හැඳින්වේ.

Question 8 [K3 - FL-4.5.3]: When using Acceptance Test-Driven Development (ATDD), in what order should the team derive and design test cases?

(ATDD), in what order should the team derive and design test cases? (Acceptance Test-Driven Development (ATDD) භාවිතයේදී, කණ්ඩායම විසින් test cases සැලසුම් කළ යුතු නිවැරදි අනුපිළිවෙළ කුමක්ද?)

A) Non-functional tests → Negative tests → Positive tests

B) Negative tests → Positive tests → White-box branch tests

C) Positive tests → Negative tests → Non-functional tests

D) Exploratory tests → Fault attacks → Acceptance tests

Correct Answer: C

Rationale (විවරණය): Section 4.5.3 ට අනුව ATDD වලදී test cases derive කරන නිවැරදි අනුපිළිවෙල වන්නේ: **Positive tests** → **Negative tests** → **Non-functional tests** යන්නයි.

Part 2: Additional Questions for Remaining Learning Objectives (ඉතිරි ඉගෙනුම් අරමුණු සඳහා අමතර ප්‍රශ්න)

Question 9 [K2 - FL-4.1.1]: A tester designs tests from personal experience of defects found in similar applications and knowledge of typical developer mistakes, without deriving the tests from a formal model or from the code structure. Which category of test techniques is this?

(සමාන යෙදුම්වල සොයාගත් දෝෂ පිළිබඳ පෞද්ගලික අත්දැකීම් සහ සංවර්ධකයින්ගේ සාමාන්‍ය වැරදි පිළිබඳ දැනුම මත පදනම්ව, විධිමත් ආකෘතියකින් හෝ code ව්‍යුහයෙන් පරීක්ෂණ ව්‍යුත්පන්න නොකර, ටෙස්ටර් කෙනෙක් පරීක්ෂණ සැලසුම් කරයි. මෙය කුමන වර්ගයේ පරීක්ෂණ තාක්ෂණයක්ද?)

A) Black-box test techniques, because the tester does not look at the code.

B) White-box test techniques, because the tester uses knowledge of defects.

C) Experience-based test techniques.

D) Static test techniques.

Correct Answer: C

Rationale (සිංහල + English):

නිවැරදි පිළිතුර C. Black-box techniques පරීක්ෂණ ව්‍යුත්පන්න කරන්නේ අභ්‍යන්තර ව්‍යුහයට යොමු නොවී specifications/ආකෘති මගිනි. White-box techniques පරීක්ෂණ ව්‍යුත්පන්න කරන්නේ code, architecture හෝ detailed design වැනි ව්‍යුහයෙනි. Experience-based techniques පරීක්ෂකවරුන්ගේ, සංවර්ධකයින්ගේ සහ පරිශීලකයින්ගේ දැනුම සහ අත්දැකීම් මත පදනම් වේ. Static techniques යනු code ක්‍රියාත්මක නොකර කරන ඇගයීමකි.

English: Black-box techniques derive tests from specifications and models without referring to the internal structure. White-box techniques derive tests from the structure (code, architecture, detailed design). Experience-based techniques use the knowledge and experience of testers, developers, users and other stakeholders. Static techniques evaluate work products without executing code (Section 4.1).

Question 10 [K3 - FL-4.2.3]: A decision table for ATM cash withdrawal uses the conditions C1 "Card valid?", C2 "PIN correct?" and C3 "Sufficient balance?". Rules: R1 (C1=Y, C2=Y, C3=Y) dispense cash; R2 (Y, Y, N) show "insufficient funds"; R3 (Y, N, –) reject the PIN; R4 (N, –, –) retain the card. The "–" means the condition is irrelevant for that rule. What is the minimum number of test cases needed to achieve 100% decision table coverage?

(ATM මුදල් ආපසු ගැනීම සඳහා decision table එකක conditions: C1 "කාඩ්පත වලංගුද?", C2 "PIN නිවැරදිද?", C3 "ප්‍රමාණවත් ශේෂයක් තිබේද?". රීති: R1 (Y,Y,Y) මුදල් නිකුත් කරයි; R2 (Y,Y,N) "ශේෂය ප්‍රමාණවත් නොවේ" පණිවිඩය; R3 (Y,N,–) PIN ප්‍රතික්ෂේප කරයි; R4 (N,–,–) කාඩ්පත රඳවා ගනී ("–" යනු එම රීතියට එම condition එක අදාළ නොවන බවයි). 100% decision table coverage ලබා ගැනීමට අවශ්‍ය අවම පරීක්ෂණ ගණන කොපමණද?)

A) 3

B) 4

C) 5

D) 8

Correct Answer: B

Rationale (සිංහල + English):
නිවැරදි පිළිතුර B (4). Decision table coverage = අවම වශයෙන් එක් පරීක්ෂණයකින් ආවරණය කළ rules (තීරු) ගණන ÷ ශක්‍ය rules මුළු ගණන. "–" (irrelevant) භාවිතයෙන් rules 4 ක් දක්වා සංක්ෂිප්ත කර ඇති නිසා, සෑම rule එකක් සඳහාම එක් පරීක්ෂණයක් බැගින් **පරීක්ෂණ 4ක්** අවශ්‍යයි. සංක්ෂිප්ත නොකළ සම්පූර්ණ වගුවක තීරු $2^3 = 8$ක් තිබෙනු ඇත.
English: Decision table coverage = number of rules (columns) exercised by at least one test case / total number of feasible rules. The table has 4 rules (the "–" values collapse the $2^3 = 8$ possible combinations into 4 rules), so one test per rule gives 100% coverage: **4 test cases** (Section 4.2.3).

Question 11 [K2 - FL-4.3.1]: Which statement about statement testing is correct?

(Statement testing පිළිබඳ නිවැරදි ප්‍රකාශය කුමක්ද?)

A) 100% statement coverage means every executable statement was executed at least once, but a decision's False outcome may never have been exercised.

B) 100% statement coverage guarantees that every outcome of every decision was exercised.

C) Statement coverage is the percentage of test cases that executed without failure.

D) Statement testing can only be applied to specifications, not to source code.

Correct Answer: A

Rationale (සිංහල + English):

නිවැරදි පිළිතුර A. Statement coverage = පරීක්ෂණ මගින් ක්‍රියාත්මක කළ ප්‍රකාශ (statements) ගණන ÷ ක්‍රියාත්මක කළ හැකි මුළු ප්‍රකාශ ගණන (ප්‍රතිශතයක් ලෙස). 100% statement coverage යනු සෑම ක්‍රියාත්මක කළ හැකි ප්‍රකාශයක්ම අවම වශයෙන් එක් වරක් ක්‍රියාත්මක වූ බවයි; නමුත් decision එකක "False" ප්‍රතිඵලය කිසිවිටෙක පරීක්ෂා නොවිය හැකි බැවින් එය 100% branch coverage සහතික නොකරයි.

English: Statement coverage = number of statements executed by the tests / total number of executable statements (as a percentage). With 100% statement coverage every executable statement has been executed at least once, but this does not guarantee that all decision outcomes were tested (that requires branch coverage) and it may not detect defects that depend on particular data (Section 4.3.1).

Question 12 [K2 - FL-4.4.1]: A tester prepares a list of likely faults from experience (division by zero, empty input, leap-year dates, null values) and designs tests to trigger them. Which technique is this?
(ටෙස්ටර් කෙනෙක් අත්දැකීම් මත පදනම්ව ඇතිවිය හැකි දෝෂ ලැයිස්තුවක් (බිංදුවෙන් බෙදීම, හිස් ආදාන, සමවත්සර දින, null අගයන්) සකසා ඒවා අවුලුවන පරීක්ෂණ සැලසුම් කරයි. මෙය කුමන තාක්ෂණයද?)

A) Boundary value analysis, because the tests target typical limits.

B) State transition testing, because the tester models states.

C) Statement testing, because the tester examines the code.

D) Error guessing (using a fault attack), because the tester anticipates defects from experience and designs tests to expose them.

Correct Answer: D

Rationale (සිංහල + English):

නිවැරදි පිළිතුර D. Error guessing යනු ටෙස්ටර්වරයා දෝෂ, වැරදි සහ අසාර්ථකත්වයන් පෙරකිය හැකි ලෙස — යෙදුම අතීතයේ ක්‍රියාකළ ආකාරය, සංවර්ධකයින් කරන වැරදි වර්ග සහ වෙනත් යෙදුම්වල අසාර්ථකත්වයන් පිළිබඳ දැනුම මත පදනම්ව — පරීක්ෂණ සැලසුම් කිරීමයි. ක්‍රමානුකූල ක්‍රමයක් වන fault attack එකේදී ඇතිවිය හැකි දෝෂ ලැයිස්තුවක් සාදා ඒවා හෙළිදරව් කරන පරීක්ෂණ සැලසුම් කරයි.

English: Error guessing is a technique in which testers anticipate errors, defects and failures based on their knowledge: how the application has worked in the past, the types of errors developers make, and failures seen in other applications. A systematic approach is the fault attack: list possible errors, defects and failures, then design tests that expose them (Section 4.4.1).

Question 13 [K2 - FL-4.4.3]: Which statement about checklist-based testing is TRUE?

(Checklist-based testing පිළිබඳ සත්‍ය ප්‍රකාශය කුමක්ද?)

A) Checklist items should be very general so that the checklist never needs updating.

B) A checklist can be derived from experience, from knowledge of what matters to users and from knowledge of how software fails; its items should be phrased so that they can be checked independently, often as questions.

C) The checklist must be completed once before testing starts and must never be changed.

D) Checklist-based testing provides precise, repeatable coverage metrics.

Correct Answer: B

Rationale (සිංහල + English):

නිවැරදි පිළිතුර B. Checklist එක අත්දැකීම්, පරිශීලකයාට වැදගත් දේ පිළිබඳ දැනුම සහ මෘදුකාංග අසාර්ථක වන ආකාරය පිළිබඳ අවබෝධය මත ගොඩනැගිය හැකිය; එහි අයිතම ප්‍රශ්න ආකාරයෙන්, එකිනෙකින් ස්වාධීනව සත්‍යාපනය කළ හැකි ලෙස ලිවිය යුතුය. Checklist නිතිපතා යාවත්කාලීන කළ යුතුය; එයට ස්වයංක්‍රීය මෙවලමකින් පරීක්ෂා කළ හැකි හෝ අධික ලෙස පොදු අයිතම ඇතුළත් නොකළ යුතුය. Checklist ඉහළ මට්ටමේ (high-level) බැවින් පුනරාවර්තන හැකියාව (repeatability) සහ මනින ලද ආවරණය සීමිතය.

English: Checklists are built from experience, user priorities and understanding of why and how software fails. Good items are phrased as questions, are independently verifiable, do not overlap and are not too general; items that automated tools can check or that are better as entry/exit criteria should be excluded. Checklists should be updated regularly. Because checklists are high-level, repeatability and measurable coverage are limited (Section 4.4.3).

Question 14 [K2 - FL-4.5.1]: Which of the following is a well-formed user story?

(පහත සඳහන් ඒවායින් නිවැරදිව ගොඩනගන ලද user story එක කුමක්ද?)

A) "The system shall store passwords in a relational database."

B) "Implement the password reset service using SHA-256 during sprint 3."

C) "As a registered customer, I want to reset my password by e-mail so that I can regain access without calling support."

D) "Testers must manually verify the reset feature before every release."

Correct Answer: C

Rationale (සිංහල + English):

නිවැරදි පිළිතුර C. User story එකක මූලික අංග 3ක් ඇත: භූමිකාව (role), අරමුණ/ක්‍රියාකාරිත්වය (goal), සහ ව්‍යාපාරික වටිනාකම (business value). හොඳ user story එක INVEST (Independent, Negotiable, Valuable, Estimable, Small, Testable) ගුණාංගවලට අනුකූල විය යුතු අතර 3 C's (Card, Conversation, Confirmation) අනුගමනය කරයි. සංවර්ධකයින්, ව්‍යාපාරික නියෝජිතයින් සහ ටෙස්ටර්වරුන් එක්ව ලිවීමෙන් story එක පරීක්ෂා කළ හැකි (testable) වේ.

English: A user story has three key elements: the role, the goal (functionality) and the business value. Good user stories follow INVEST (Independent, Negotiable, Valuable, Estimable, Small, Testable) and the 3 Cs: Card, Conversation, Confirmation. Writing stories collaboratively with developers, business representatives and testers makes them testable (Section 4.5.1).

Question 15 [K2 - FL-4.5.2]: The acceptance criterion "Given a registered user with a valid password, When the user logs in, Then the dashboard is displayed" is written in which format?

("ලියාපදිංචි පරිශීලකයෙකු වලංගු මුරපදයක් සහිතව සිටී (Given), ඔහු ලොග් වේ (When), එවිට dashboard එක ප්‍රදර්ශනය වේ (Then)" යන පිළිගැනීමේ නිර්ණායකය (acceptance criterion) ලියා ඇත්තේ කුමන ආකෘතියෙන්ද?)

A) Scenario-oriented acceptance criteria (Given/When/Then).

B) Rule-oriented acceptance criteria written as a bullet list of rules.

C) Rule-oriented acceptance criteria written as an input/output table.

D) It is not an acceptance criterion; it is a unit test case.

Correct Answer: A

Rationale (සිංහල + English):

නිවැරදි පිළිතුර A. Acceptance criteria විකල්ප: scenario-oriented (Given/When/Then — BDD හි භාවිත වන), rule-oriented (සත්‍යාපන රීති ලැයිස්තුවක් හෝ ආදාන-ප්‍රතිදාන වගුවක්), සහ පැහැදිලිව නිර්වචනය කළ, අපැහැදිලි නොවන වෙනත් ආකෘති. ලබාදී ඇති උදාහරණය Given/When/Then ආකාරයෙන් ඇති නිසා එය scenario-oriented වේ.

English: Acceptance criteria can be scenario-oriented (Given/When/Then, as used in BDD), rule-oriented (a bullet list of verification rules or an input/output table) or written in another format, as long as it is well defined and unambiguous. The example uses Given/When/Then, so it is scenario-oriented (Section 4.5.2).